42. Trapping Rain Water Hard

Acceptance: 66.6% | LeetCode

Given `n` non-negative integers representing an elevation map where the width of each bar is `1`, compute how much water it can trap after raining.

Example 1:

```
**Input:** height = [0,1,0,2,1,0,1,3,2,1,2,1]
**Output:** 6
**Explanation:** The above elevation map (black section) is represented by array [0,1,0,2,1,0,1,3,2,1,2,1]. In this case, 6 units of rain water (blue section) are being trapped.
```

Example 2:

```
**Input:** height = [4,2,0,3,2,5]
**Output:** 9
```

Constraints:

- `n == height.length`
- `1 <= n <= 2 * 104`
- `0 <= height[i] <= 105`

Claude Opus Solution

Trapping Rain Water — Complete Solution Guide

1. Intuition

The **key insight** is to think about water trapped **column by column**, not as a whole.

> For any single position `i`, how much water can sit on top of it?

Water at position `i` is bounded by the **tallest wall to its left** and the **tallest wall to its right**. Water can only rise as high as the **shorter** of these two walls (otherwise it spills over).

```
water[i] = min(maxLeft[i], maxRight[i]) − height[i]
```

If this value is negative (the bar is taller than the lower wall), no water is trapped there.

Visual example for `height = [4,2,0,3,2,5]`:

```
At index 2 (height=0): leftMax=4, rightMax=5 → min(4,5)−0 = 4 units
```

The challenge is computing `maxLeft` and `maxRight` **efficiently** for every position.

---

2. Approach

I'll present the **Two Pointers** solution as the optimal one (O(1) space).

```
KEY OBSERVATION:
- We only need min(leftMax, rightMax).
- If leftMax < rightMax, water at left is determined SOLELY by leftMax
  (we KNOW rightMax is bigger, so it doesn't matter what its exact value is).

PSEUDOCODE:
1. Set two pointers: left=0, right=n−1
2. Track leftMax and rightMax seen so far
3. While left <= right:
     a. If leftMax <= rightMax:
          − leftMax is the limiting factor → safely compute water at left
          − add max(leftMax − height[left], 0)
          − update leftMax, move left++
     b. Else (rightMax < leftMax):
          − rightMax is limiting → compute water at right
          − add max(rightMax − height[right], 0)
          − update rightMax, move right−−
4. Return total volume
```

---

3. Python Solution

```python
from typing import List


class Solution:
    def trap(self, height: List[int]) -> int:
        # Two-pointer technique with O(1) space.

        left, right = 0, len(height) - 1   # Pointers from both ends
        left_max = right_max = 0           # Tallest wall seen from each side
        volume = 0                         # Accumulated trapped water

        # Process until pointers meet
        while left <= right:
            # The crucial decision: which side is the LIMITING wall?
            # If left_max <= right_max, we GUARANTEE that there exists
            # some wall on the right >= right_max >= left_max.
            # So water at 'left' depends ONLY on left_max — we can commit safely.
            if left_max <= right_max:
                # Water trapped above height[left] = left_max - height[left]
                # (clamped to 0 if current bar is taller than left_max)
                volume += max(left_max - height[left], 0)

                # Update the left wall height after processing this column
                left_max = max(left_max, height[left])

                # Move inward from the left
                left += 1
            else:
                # Symmetric case: right_max is the limiting factor
                volume += max(right_max - height[right], 0)
                right_max = max(right_max, height[right])
                right -= 1

        return volume
```

Why is the "limiting side" logic correct? When `left_max <= right_max`, even if we haven't scanned the entire right side yet, we already KNOW the right wall is at least `right_max ≥ left_max`. Therefore `min(left_max, true_right_max) = left_max`, so we can compute water at `left` with certainty.

---

4. Complexity Analysis

Approach	Time	Space
Two Pointers	**O(n)**	**O(1)**

- **Time O(n):** Each pointer moves inward exactly once; total moves = `n`. Single pass.
- **Space O(1):** Only a few scalar variables — no auxiliary arrays.

This beats the DP approach which needs **O(n)** extra space for the `left_max` / `right_max` arrays.

---

5. Alternative Approaches

A) Dynamic Programming — O(n) time, O(n) space

Precompute prefix max (`left_max[]`) and suffix max (`right_max[]`) arrays, then sum `min(left_max[i], right_max[i]) - height[i]`. - **Easiest to understand**; directly implements the core formula. - Downside: two extra arrays.

```python
def trap(self, height: List[int]) -> int:
    n = len(height)
    left_max, right_max = [0]*n, [0]*n
    left_max[0], right_max[-1] = height[0], height[-1]

    # Prefix maximum from the left
    for i in range(1, n):
        left_max[i] = max(left_max[i-1], height[i])
    # Suffix maximum from the right
    for i in range(n-2, -1, -1):
```

```
        right_max[i] = max(right_max[i+1], height[i])

    # Each column's water = lower wall minus its own height
    return sum(min(left_max[i], right_max[i]) - height[i] for i in range(n))
```

B) Monotonic Stack — O(n) time, O(n) space

Compute water **horizontally** in layers. Maintain a decreasing stack of indices. When a taller bar appears, it forms a "container" with the element below the popped bar:

```
def trap(self, height: List[int]) -> int:
    stack, volume = [], 0
    for i in range(len(height)):
        # Current bar can hold water against the previous taller bar
        while stack and height[i] > height[stack[-1]]:
            bottom = height[stack.pop()]      # Floor of the trapped region
            if not stack:                     # No left boundary → can't trap
                break
            # Height of water = min(two walls) - floor
            bounded_height = min(height[i], height[stack[-1]]) - bottom
            width = i - stack[-1] - 1         # Distance between two walls
            volume += bounded_height * width
        stack.append(i)
    return volume
```

- Clever but harder to reason about; computes trapped water in horizontal slabs.

Pattern Summary

This is a classic **"limit determined by min of two extremes"** problem. The two-pointer technique applies whenever: - The answer depends on `min(leftBoundary, rightBoundary)`, and - You can prove which boundary is binding without full information.

The same pattern appears in **Container With Most Water (#11)** — recognizing it lets you reach O(1) space immediately.

Codex Solution

1. Intuition

For each bar `i`, the amount of water above it depends on the tallest wall to its left and the tallest wall to its right.

The water trapped at index `i` is:

```
water[i] = min(max height on left, max height on right) - height[i]
```

If this value is negative, it contributes `0`.

The key insight is:

> Water level at a position is limited by the shorter of the tallest walls on both sides.

For example:

```
height = [4, 2, 0, 3, 2, 5]
```

At index `2`, height is `0`.

- Tallest wall on the left = `4`
- Tallest wall on the right = `5`
- Water level = `min(4, 5) = 4`
- Water trapped = `4 - 0 = 4`

We can solve this using dynamic programming with prefix/suffix maximum arrays, but the most space-efficient solution uses **two pointers**.

---

2. Approach: Two Pointers

We keep two pointers:

```
left  = start of array
right = end of array
```

We also maintain:

```
left_max  = tallest bar seen so far from the left
right_max = tallest bar seen so far from the right
```

At each step:

- If `left_max <= right_max`, then the left side is the limiting boundary.
- We can safely calculate water at `left`.
- Move `left` forward.
- Otherwise, the right side is the limiting boundary.
- We can safely calculate water at `right`.
- Move `right` backward.

Why is this safe?

If `left_max <= right_max`, then we already know there is a right boundary tall enough to trap water based on `left_max`. Therefore, water at `left` depends only on `left_max`.

Similarly, if `right_max < left_max`, water at `right` depends only on `right_max`.

Pseudocode

```
function trap(height):
    left = 0
    right = len(height) - 1

    left_max = 0
    right_max = 0

    total_water = 0

    while left <= right:
        if left_max <= right_max:
            if height[left] < left_max:
                total_water += left_max - height[left]
            else:
                left_max = height[left]

            left += 1

        else:
            if height[right] < right_max:
                total_water += right_max - height[right]
            else:
                right_max = height[right]

            right -= 1

    return total_water
```

---

3. Python Solution

```python
from typing import List


class Solution:
    def trap(self, height: List[int]) -> int:
        # Two pointers starting from both ends of the elevation map
        left = 0
        right = len(height) - 1

        # left_max is the tallest bar seen so far from the left side
        # right_max is the tallest bar seen so far from the right side
        left_max = 0
        right_max = 0
```

```
        # Stores the total trapped rain water
        water = 0

        # Process bars until the two pointers cross
        while left <= right:
            # If left_max is smaller, then the water level on the left side
            # is limited by left_max.
            #
            # We can safely calculate trapped water at height[left],
            # because right_max is at least as large as left_max.
            if left_max <= right_max:
                # If the current bar is lower than left_max,
                # water can be trapped above it.
                if height[left] < left_max:
                    water += left_max - height[left]
                else:
                    # Otherwise, this bar becomes the new left boundary.
                    left_max = height[left]

                # Move the left pointer inward
                left += 1

            else:
                # If right_max is smaller, then the water level on the right side
                # is limited by right_max.
                #
                # We can safely calculate trapped water at height[right],
                # because left_max is larger than right_max.
                if height[right] < right_max:
                    water += right_max - height[right]
                else:
                    # Otherwise, this bar becomes the new right boundary.
                    right_max = height[right]

                # Move the right pointer inward
                right -= 1

        return water
```

---

4. Complexity Analysis

Time Complexity: `O(n)`

Each bar is processed at most once.

- The `left` pointer only moves forward.
- The `right` pointer only moves backward.
- Together, they scan the array one time.

Therefore, the total time complexity is:

```
O(n)
```

Space Complexity: `O(1)`

We only use a few variables:

```
left, right, left_max, right_max, water
```

No extra arrays or stacks are used.

Therefore, the space complexity is:

```
O(1)
```

---

5. Alternative Approaches

Approach 1: Dynamic Programming / Prefix-Suffix Maximums

For every index, precompute:

```
left_max[i]  = tallest bar from index 0 to i
right_max[i] = tallest bar from index i to n - 1
```

Then:

```
water += min(left_max[i], right_max[i]) - height[i]
```

Complexity

```
Time:  O(n)
Space: O(n)
```

This approach is easier to understand but uses extra memory.

Approach 2: Monotonic Stack

Use a stack to store indices of bars in decreasing height order.

When we find a bar taller than the stack top, it may form a right boundary. We pop the middle bar and calculate water using:

```
bounded_height = min(left_boundary, right_boundary) - bottom_height
width = right_index - left_index - 1
water += bounded_height * width
```

Complexity

```
Time:  O(n)
Space: O(n)
```

This approach is useful when thinking of trapped water as horizontal layers between boundaries.

Key Pattern

This problem is a classic example of:

```
Two Pointers + Running Maximums
```

The main idea is to process the side with the smaller known maximum because that side determines the current possible water level.